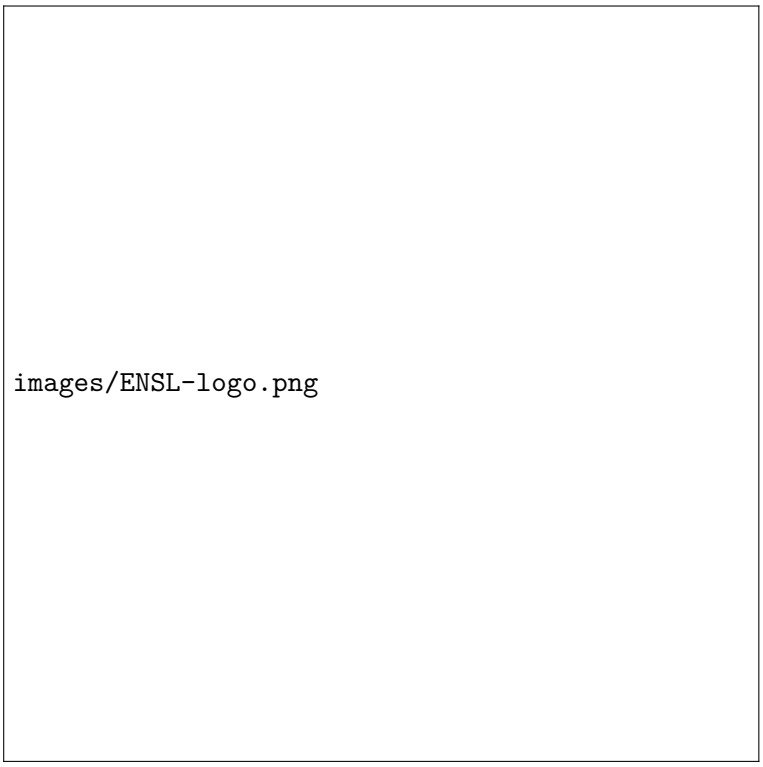
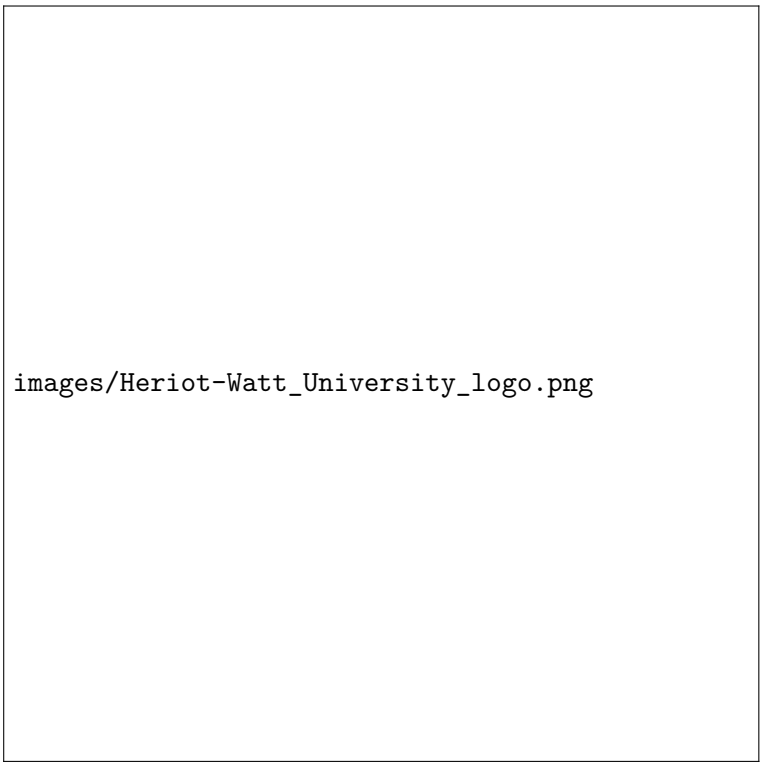


ENS de Lyon
Computer Science Department

Internship Report

Unfolded proximal networks as Flow Matching velocity fields

Name: Collas Estéban
Program: M1 Computer Science
Academic Year: 2025–2026
University Supervisor: Elisa Ricietti
Lab Tutor: Audrey Repetti



Host Laboratory: Heriot-Watt University
Internship Period: From May 4 to July 30, 2026

Done in Helsinki, on August 21, 2026

Abstract

Flow Matching (FM) achieves state-of-the-art performance in image generation, and its links with diffusion and denoising are now well described in the literature. Exploiting this equivalence, we study unfolded (or unrolled) neural networks (UNN) — light and robust architectures obtained by unrolling proximal algorithms designed for Gaussian denoising — as velocity estimators for FM. We show that an unfolded strongly-convex Chambolle–Pock network, once conditioned on the noise level, generates samples on 2-moons, MNIST and [CIFAR-10]. Beyond sample quality, we argue that such architectures bring an interpretability that is out of reach of the usual backbones: because they alternate between a primal variable that remains a full-resolution image and a dual variable, every internal state of the network can be displayed, and the denoising performed at each layer of each integration step can be monitored directly. We finally discuss the limitations of unfolded architectures for generation, in particular the expressivity cost of keeping the primal iterate in image space.

Keywords. Flow Matching, unrolled neural networks, proximal algorithms, primal–dual splitting, Chambolle–Pock, generative modelling, interpretability.

MSC. 68T07, 49M29, 90C25, 94A08.

Contents

1 Introduction

Generative modelling of images is increasingly dominated by two closely related families of methods: diffusion models and Flow Matching (FM). Both build a transport between a simple reference distribution (typically a standard Gaussian) and the data distribution, and both reduce, at the level of a single time step, to a denoising problem: predicting a clean signal from a noisy observation. This denoising equivalence, made precise for FM by [8, 7], is the starting point of this internship.

Almost all FM and diffusion backbones used in practice are large convolutional or attention-based black boxes (UNets, DiTs), chosen for their raw sample quality rather than for any structural relationship to the underlying denoising problem. In parallel, the signal-processing community has developed *unrolled* (or *unfolded*) neural networks: architectures obtained by truncating a convergent proximal algorithm for a variational denoising problem and letting its parameters be learned end to end [14, 9]. These networks are considerably lighter than standard CNNs at comparable denoising performance, and inherit Lipschitz robustness certificates from the optimisation literature [11, 15].

The question addressed by this internship is whether this second family – interpretable, certifiable, but so far confined to inverse problems such as Gaussian denoising – can be used as the velocity estimator of an FM generative model, and at what cost in generation quality. This is, to our knowledge, an unexplored combination at the start of the internship (Section 3): UNNs had been studied extensively for denoising, and FM extensively with black-box backbones, but not together.

Our contribution (Section 4) is threefold. First, we make the denoising/FM equivalence operational: given any proximal denoiser, we describe exactly how to turn it into an FM velocity field, and how to condition it on the noise level. Second, we instantiate this with an unrolled strongly-convex Chambolle–Pock network (ConvScCP) and study it on 2-moons, MNIST and [CIFAR-10], comparing it to UNet baselines under matched Flow Matching training. Third, we show that the architecture is genuinely interpretable – many internal states are displayable images – and use this to observe how the network denoises internally, while also identifying and correcting a structural limitation (an odd-symmetry constraint) and reporting the negative results that shaped the final design (Section 8.1.1). We close (**????**) with a discussion of the expressivity trade-off this interpretability comes with, and directions for future work.

2 Preliminaries

This section collects the background material – convex analysis, proximal algorithms, unrolled networks and Conditional Flow Matching – needed to state our contribution. None of the results in this section are original; each is attributed to its source.

2.1 Convex Analysis

In this section, we summarize the fundamental mathematical tools and definitions from convex analysis and vector calculus required to formalize unrolled architectures and flow matching objectives. Throughout this document, $\mathcal{H}$ denotes a real Hilbert space.

Definition 2.1 (The class Γ_0) We denote by $\Gamma_0(\mathcal{H})$ the set of all proper, lower-semicontinuous, convex functions mapping $\mathcal{H}$ to $]-\infty, +\infty]$. A function g is *proper* if its domain $\operatorname{dom}(g) = \{x \in \mathcal{H} \mid g(x) < +\infty\}$ is non-empty.

Definition 2.2 (Fenchel-Legendre Conjugate) Let $g \in \Gamma_0(\mathcal{H})$. The Fenchel-Legendre conjugate of g is the function $g^* \in \Gamma_0(\mathcal{H})$ defined by:

$$g^*(u) = \sup_{x \in \mathcal{H}} \left(\langle x \mid u \rangle - g(x) \right). \tag{2.1}$$

Example 2.3 (Quadratic Function) Let $g(x) = \frac{1}{2}\|x\|^2$. Its conjugate is:

$$g^*(u) = \frac{1}{2}\|u\|^2. \tag{2.2}$$

More generally, if $g(x) = \frac{1}{2}\langle Ax \mid x \rangle$ where A is a continuous symmetric positive definite operator, then $g^*(u) = \frac{1}{2}\langle u \mid A^{-1}u \rangle$.

Example 2.4 (Norms and Dual Norms) More generally, let $g(x) = \|x\|$ be a norm on $\mathcal{H}$. Its conjugate is the indicator function of the unit ball of the dual norm $\|\cdot\|_*$:

$$g^*(u) = \delta_{\mathcal{B}_*}(u) = \begin{cases} 0 & \text{if } \|u\|_* \leqslant 1, \\ +\infty & \text{otherwise,} \end{cases} \tag{2.3}$$

where $\|u\|_* = \sup\{\langle u \mid x \rangle : \|x\| \leqslant 1\}$.

Example 2.5 (Indicator and Support Functions) Let $C \subseteq \mathcal{H}$ be a non-empty closed convex set. Let $g = \delta_C$ be the indicator function of C (i.e., $g(x) = 0$ if $x \in C$ and $+\infty$ otherwise). Its conjugate is the support function of C:

$$g^*(u) = \sigma_C(u) = \sup_{x \in C} \langle u \mid x \rangle. \tag{2.4}$$

Definition 2.6 (Proximal Operator) Let $g \in \Gamma_0(\mathcal{H})$. For any $\gamma \in]0, +\infty[$, the proximity operator of γg is the unique mapping $\text{prox}_{\gamma g} : \mathcal{H} \to \mathcal{H}$ defined as:

$$\text{prox}_{\gamma g}(x) = \underset{y \in \mathcal{H}}{\text{argmin}} \quad \left(g(y) + \frac{1}{2\gamma} \|y - x\|^2 \right). \tag{2.5}$$

Proposition 2.7 (Moreau Decomposition) *Let $g \in \Gamma_0(\mathcal{H})$ and $\gamma \in]0, +\infty[$. For any $x \in \mathcal{H}$, the following identity holds:*

$$\text{prox}_{\gamma g}(x) = x - \gamma \text{prox}_{\gamma^{-1} g^*}(\gamma^{-1} x). \tag{2.6}$$

This formula is central to unrolling algorithms in the dual domain, as it allows expressing a proximal step as a residual connection (see [???]).

2.2 Vector Calculus and Dynamical Systems

The study of Continuous Normalizing Flows and Flow Matching requires defining operators on vector fields $v : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$ and probability densities $p_t : \mathbb{R}^d \to \mathbb{R}$.

Definition 2.8 (Divergence) The divergence of a differentiable vector field $v(x) = (v_1(x), \ldots, v_d(x))$ is the scalar function defined by:

$$\text{div}(v) = \nabla \cdot v = \sum_{i=1}^{d} \frac{\partial v_i}{\partial x_i}. \tag{2.7}$$

Definition 2.9 (Continuity Equation) A probability path $p_t(x)$ is generated by a time-dependent vector field $v_t(x)$ if it satisfies the continuity equation (also known as the transport equation):

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x) v_t(x)) = 0. \tag{2.8}$$

2.3 Linear Algebra and Stability

Definition 2.10 (Spectral Norm) For a linear operator $L : \mathbb{R}^N \to \mathbb{R}^M$, the spectral norm (or induced L_2 norm) is defined as:

$$\|L\|_S = \sup_{x \neq 0} \frac{\|Lx\|_2}{\|x\|_2} = \sqrt{\lambda_{\max}(L^\top L)}, \tag{2.9}$$

where $\lambda_{\max}$ denotes the largest eigenvalue. *This norm is used to provide certificates of robustness and convergence for unrolled neural networks (see [??]).*

2.4 Feedforward Networks

Feedforward Neural Networks (FNNs), or Multi-Layer Perceptrons (MLPs), represent the most fundamental architecture in deep learning. Their primary characteristic is that information flows in a single direction: from the input layer, through one or more hidden layers, to the output layer, without any cycles or loops.

Mathematical Formulation. An FNN can be viewed as a function $\mathcal{F}_\Theta$ that maps an input $x_0 \in \mathbb{R}^{d_{in}}$ to an output $x_K \in \mathbb{R}^{d_{out}}$ through a sequence of K layer transformations. Each layer $k \in \{1, \dots, K\}$ performs an affine transformation followed by a non-linear activation function σ_k:

$$x_k = \sigma_k(W_k x_{k-1} + b_k) \tag{2.10}$$

where:

- $W_k \in \mathbb{R}^{N_k \times N_{k-1}}$ is the **weight matrix** of the k-th layer.
- $b_k \in \mathbb{R}^{N_k}$ is the **bias vector** (or shift parameter).
- σ_k is the **activation function** (not necessarily applied element-wise), which introduces the non-linearity necessary for the network to approximate complex, non-convex functions. Common choices include ReLU ($= \max(0, .)$), Sigmoid, or GELU, applied element-wise.

The complete network is thus a composition of operators:

$$\mathcal{F}_\Theta(x_0) = (L_K \circ L_{K-1} \circ \cdots \circ L_1)(x_0) \tag{2.11}$$

where $L_k(x) = \sigma_k(W_k x + b_k)$, and the set of all learnable parameters is $\Theta = \{W_k, b_k\}_{k=1}^K$.

2.5 Proximal Algorithms

Proximal algorithms come from the so-called "variational approach", which consists of adding a prior to an optimisation problem in order to solve it. More concretely, the starting point of the variational approach is typically an inverse problem, where one seeks to recover an underlying signal $\bar{x}$ from a degraded measurement z. We will consider the case where:

$$z = A\bar{x} + \epsilon, \tag{2.12}$$

with A a linear operator, and $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

Without using training data, but knowing A, a natural way to recover $\bar{x}$ would be to solve:

$$\min_x \underbrace{\frac{1}{2}\|z - Ax\|_2^2}_{f_z(x)} \tag{2.13}$$

However, this kind of problem is almost always ill-posed (A is not injective). Hence, to get a better solution (e.g. a realistic denoised image), we add a prior g on x (ie., we use the variational approach). A common form for the prior leads to:

$$\min_x \underbrace{\frac{1}{2}\|z - Ax\|_2^2}_{f_z(x)} + \underbrace{\mu\varphi(Wx)}_{g(x)} \tag{2.14}$$

with $\varphi \in \Gamma_0(\mathcal{H})$, W a linear operator (ie. $W \in \mathcal{L}(\mathcal{H})$), $\mu \geqslant 0$. Here, f_z is the data fidelity term and $g \in \Gamma_0(\mathcal{H})$ is a regularization term (prior) promoting specific properties (like sparsity if φ is the ℓ_1 norm). The operator W is often a sparsifying transform (e.g., wavelets or finite differences). μ indicates how much regularization we want.

Problems such as (2.14) are often solved using iterative algorithms of the form:

$$x_{k+1} = \mathcal{T}_k(x_k, z; \Theta) \tag{2.15}$$

where $\mathcal{T}_k$ are transition operators and Θ is a set of fixed (hyper)parameters (e.g., step sizes, regularization weights). Given appropriate assumptions, those algorithms are designed to converge to a solution of (2.14) as $k \to \infty$.

When g is non-differentiable (eg. if φ is the ℓ_1 norm), standard gradient descent cannot be applied to solve (2.14). Instead, the iterative algorithms we use contain proximal operators (see 2.6). Hence, they are called "proximal algorithms".

2.5.1 Some Proximal Algorithms

Let $f, f_1, f_2, \varphi, g \in \Gamma_0(\mathcal{H})$ and $W \in \mathcal{L}(\mathcal{H})$.

Forward-Backward (FB) Algorithm. The FB algorithm [5] is the standard method for solving $\min_x f(x) + g(x)$ (see (2.14)) when f is convex and L-smooth (i.e., ∇f is L-Lipschitz). The update rule is:

$$x_{k+1} = \mathrm{prox}_{\gamma g}(x_k - \gamma \nabla f(x_k)) \tag{2.16}$$

where γ is the step size. If $\gamma \in (0, 2/L)$, x_k converges weakly to a minimizer, and the objective value converges at a rate of $O(1/k)$.

Dual Forward-Backward (Dual FB). When the prior is complex (e.g., $g = \mu \cdot \varphi \circ W$ where W is not easily invertible), it is often more efficient to solve the dual problem. The classical form of Dual FB iterations [11], for $\min_x f(x) + \frac{1}{2}\|x - z\|^2 + \varphi(Wx)$, leads to:

$$\begin{cases} x_{k+1} = \mathrm{prox}_f(z - W^* u_k) \\ u_{k+1} = \mathrm{prox}_{\tau\varphi^*}(u_k + \tau W x_{k+1}) \end{cases} \tag{2.17}$$

where φ^* is the Fenchel conjugate of φ, and $\tau > 0$. Convergence of x_k is guaranteed if $\tau \in (0, 2/\|W\|^2)$.

Chambolle-Pock (CP) Algorithm. The Chambolle-Pock algorithm [3] is designed for problems of the form $\min_x f(x) + \varphi(Wx)$, where both f and φ are potentially non-smooth but have simple proximal operators. The iterations are:

$$\begin{cases} u_{k+1} = \mathrm{prox}_{\sigma\varphi^*}(u_k + \sigma W \tilde{x}_k) \\ x_{k+1} = \mathrm{prox}_{\tau f}(x_k - \tau W^* u_{k+1}) \\ \tilde{x}_{k+1} = 2x_{k+1} - x_k \end{cases} \tag{2.18}$$

The algorithm converges to a saddle point of the primal-dual problem provided that the step sizes satisfy $\tau\sigma\|W\|^2 < 1$. Unlike FB, CP does not require any function to be differentiable.

Strongly-Convex Chambolle–Pock (ScCP). The ScCP algorithm [3] is the accelerated variant of CP, exploiting the strong convexity of the data-fidelity term $f_z(x) = \frac{1}{2}\|z - x\|^2$. Adapting the formulation from [11] to our unconstrained setting and denoting the primal and dual stepsizes as τ_k and σ_k respectively, the associated iterations read, for $k = 0, 1, \ldots$:

$$\begin{cases} x_{k+1} = \dfrac{1}{1+\tau_k}(x_k - \tau_k W^\top u_k) + \dfrac{\tau_k}{1+\tau_k} z, \\ u_{k+1} = \mathrm{prox}_{\sigma_k g^*}\left(u_k + \sigma_k W\left[(1+\alpha_k)\, x_{k+1} - \alpha_k\, x_k\right]\right), \end{cases} \tag{2.19}$$

where $x_0 = z$, $u_0 = 0$, and α_k is a momentum parameter. Note that when $\alpha_k = 1$, (2.19) reduces to standard CP; and when $\alpha_k = 0$, it recovers the Arrow–Hurwicz algorithm [1].

In the accelerated ScCP scheme, the momentum schedule is driven by the strong convexity constant of f_z, which is exactly 1; we therefore fix $\rho = 1$ throughout, so that $\alpha_k = (1 + 2\tau_k)^{-1/2}$. If $(\tau_k)_{k\in\mathbb{N}}$ and $(\sigma_k)_{k\in\mathbb{N}}$ are positive sequences satisfying $\tau_0\sigma_0\|W\|_2^2 \leqslant 1$, with $\tau_{k+1} = \alpha_k\tau_k$ and $\sigma_{k+1} = \sigma_k/\alpha_k$, the sequence converges to the exact solution $\hat{x} = \lim_{k\to\infty} x_k$ [3].

2.5.2 A simple case: ISTA

To illustrate this, consider the sparse coding problem, which aims to find a sparse vector x such that $z \approx Ax$ for a given dictionary A. This is formulated as the LASSO problem:

$$\min_x \underbrace{\frac{1}{2}\|z - Ax\|_2^2}_{f_z(x)} + \underbrace{\mu\|x\|_1}_{g(x)} \tag{2.20}$$

ISTA. The Iterative Soft-Thresholding Algorithm (ISTA) [2] solves (2.20) by alternating between a gradient descent step on the data fidelity term and a proximal mapping (soft-thresholding) for the L_1 penalty:

$$x_{k+1} = \text{prox}_{\frac{\mu}{L}||.||}\left(x_k - \frac{1}{L}A^T(Ax_k - z)\right) \tag{2.21}$$

where L is a Lipschitz constant of ∇f_z, and the soft-thresholding operator $\text{prox}_{\lambda||.||} = \text{sign}(s)\max(|s| - \lambda, 0) =: \eta_\lambda$ is applied element-wise.

Remark 2.11 In this case, the smallest Lipschitz constant of ∇f_z is $\lambda_{max}(A^TA)$.

Remark 2.12 It is FB with $W = I$ and $\varphi = \mu\|\cdot\|_1$.

Remark 2.13 We can rewrite this update as:

$$x_{k+1} = \eta_\lambda\left(W_1x_k + W_2z\right) \tag{2.22}$$

with $W_1 = I - \frac{1}{L}A^TA$, $W_2 = \frac{1}{L}A^T$ and $\lambda = \frac{\mu}{L}$.

In this form, ISTA has the form of a feed-forward neural network. This remark is the basis of Unrolled Neural Networks.

2.6 Unrolled Neural Networks (UNN)

Unrolled Neural Networks, also known as algorithm unrolling or deep unrolling, defines a class of machine learning models obtained by interpreting the iterations of an optimization algorithm as the layers of a neural network [14]. This makes them more interpretable and often more efficient than "black-box" architectures such as standard CNNs or Transformers.

Starting from an algorithm of the form of (2.15), a UNN is constructed by:

(i) **Truncation:** Fixing the algorithm to a finite number of iterations K.

(ii) **Parametrization:** Replacing the fixed parameters Θ with learnable weights θ_k for each layer k.

(iii) **End-to-End Training:** Optimizing $\Theta = \{\theta_k\}_{k=1}^K$ using backpropagation on a dataset of (z, x_{true}) pairs, using some loss appropriate to our optimization problem.

This results in a network where each layer corresponds to a single iteration of the original algorithm, but the parameters of the "solver" are optimized to work with K iterations, and to solve a different optimization problem, defined using the specific distribution of the training data (see [9], [11], [16], [14] for more details). For Gaussian denoising, such networks reach performances comparable to established denoising networks with orders of magnitude fewer parameters, while admitting Lipschitz certificates of robustness [11, 15].

2.6.1 Example: LISTA

For example, if one wants to recover $\bar{x}$ from (2.12) using training data $(x_i, z_i)_i$, a natural loss is the MSE loss:

$$\mathcal{L}_{MSE}(\hat{x}) = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{2}||x_i - \hat{x}(z_i)||_2^2\right) \tag{2.23}$$

Remark 2.14 An optimal estimator for (2.23) is a maximum likelihood estimator, in the case $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

From here, one could use any architecture to model $\hat{x}(.)$. An appropriate (see remark 2.15) UNN would be LISTA:

LISTA. The *Learned Iterative Soft-Thresholding Algorithm* (LISTA) [9] transforms each ISTA iteration into a neural network layer. Starting from ISTA seen as in (2.22), instead of fixing W_1 and W_2 based on the matrix A, LISTA treats them as learnable weight matrices. LISTA also learns λ:

$$x_{k+1} = \eta_{\lambda_k}\left(W_{1,k}x_k + W_{2,k}z\right) \tag{2.24}$$

Using this architecture, the network can now learn the optimal operators to reconstruct x from z, by solving 2.23 [11].

In other words, for the learning process, we use the MSE loss:

$$\mathcal{L}(\Theta) = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{2}||x_i - \text{LISTA}_\Theta(z_i)||_2^2\right) \tag{2.25}$$

with (x_i, z_i) our data points and $\Theta = \{\lambda_k, W_{1,k}, W_{2,k}\}$.

In general, unrolled algorithms do not return the convergence point of the iterative algorithm they are based on, because they are trained to solve a loss different than 2.14. However, when using ISTA's convergence points as "true" data points x_i, LISTA is roughly 20 times faster than FISTA for approximate solutions [9]. Also, because the architecture is constrained by the physics of the problem, it generalizes better than pure black-box models, especially when the amount of training data is limited.

There are many possible implementations of UNN, for example if $A = BC$, we could choose to keep B and to learn only C, or even learning C^T independently from C, so introducing a mismatched adjoint (we already had a mismatched adjoint in the above example). We could also replace η_λ by a more general neural network $\mathfrak{D}_\lambda$ (see [16] for more details).

Remark 2.15 LISTA is "appropriate" for this optimization problem because ISTA solves a similar problem (see 2.5.2). We can view LISTA as a way to learn the prior g in ISTA.

2.7 Conditional Flow Matching (CFM)

this section was heavily inspired by [8].

Conditional Flow Matching (CFM) [13] is a simulation-free method for training Continuous Normalizing Flows (CNFs). It overcomes the primary bottleneck of standard CNFs, which is the need to solve an Ordinary Differential Equation (ODE) during each training iteration, by regressing a vector field against analytically defined conditional vector fields.

2.7.1 Background: Continuous Normalizing Flows

Let's say we want to sample from an unknown distribution p_1, using data points generated following p_1 (e.g., images of cats). CNFs allow us to do this sampling by following a probability path between a known distribution p_0 (e.g., a standard Gaussian) and p_1 [4].

More precisely, a CNF learns a vector field $v_t(x) : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ that generates a probability path p_t between p_0 and p_1 via the continuity equation:

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (p_t(x)v_t(x)) \tag{2.26}$$

There are many paths between p_0 and p_1. When we use CFM for training a CNF, we start by choosing one of them (see Section 2.7.4). For now, we consider p_0, v_t fixed. In theory, given p_0 and v_t, one can use integration schemes, such as the Euler scheme, to sample from p_1. However, v_t is often intractable.

The goal of flow matching is to find a neural network $v_\theta(x,t)$ that approximates the chosen vector field $v_t(x)$ (hence allowing us to use integration schemes).

2.7.2 The Flow Matching Objective

The ideal objective would be to minimize the Flow Matching (FM) loss:

$$\mathcal{L}_{\text{FM}}(\theta) := \mathbb{E}_{t\sim\mathcal{U}[0,1],x\sim p_t(x)}\left[\|v_\theta(x,t) - v_t(x)\|^2\right] \tag{2.27}$$

However, this objective is intractable because we typically do not have access to the marginal density $p_t(x)$ nor the marginal vector field $v_t(x)$ in closed form.

2.7.3 The Conditional Approach

CFM circumvents this by introducing a conditioning variable $z \sim q(z)$ independent of t (most commonly a data sample $z = x_1 \sim p_1$ or $z = (x_0, x_1) \sim p_0 \times p_1$, where we approximate p_1 with p_{data} in practice). We define a *conditional probability path* $p_t(x|z)$ and a corresponding *conditional vector field* $v_t(x|z)$ that satisfy the following equations:

$$\forall x, \mathbb{E}_z[p(x|z, t = 0)] = p_0(x) \tag{2.28}$$

$$\forall x, \mathbb{E}_z[p(x|z, t = 1)] = p_1(x) \tag{2.29}$$

$$\frac{\partial p_t(x|z)}{\partial t} = -\nabla \cdot (p_t(x|z)v_t(x|z)) \tag{2.30}$$

By aggregating these conditional paths, we recover the marginal distribution:

$$p_t(x) := \int p_t(x|z)q(z)dz \tag{2.31}$$

Crucially, it has been proven [8] that under the hypotheses above, the marginal vector field $v_t(x)$ can be expressed as:

$$v_t(x) = \mathbb{E}_{z|(x,t)}[v_t(x|z)] \tag{2.32}$$

Hence, choosing $q(z)$ and $p_t(x|z)$ amounts to choosing the vector field $v_t(x)$.

Furthermore, this leads to the **Conditional Flow Matching** objective:

$$\mathcal{L}_{\text{CFM}}(\theta) := \mathbb{E}_{t\sim\mathcal{U}[0,1],z\sim q(z),x\sim p_t(x|z)}\left[\|v_\theta(x,t) - v_t(x|z)\|^2\right] \tag{2.33}$$

One can show [8] that

$$\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + \text{const.} \tag{2.34}$$

Hence, we can minimize the intractable FM loss by minimizing the CFM loss, provided we can define $p_t(x|z)$ and $v_t(x|z)$ analytically. Now, all that remains is to choose the right conditional path.

2.7.4 Gaussian Paths

A simple choice for the conditional path is to use dirac distributions [8]. Given a noise sample $x_0 \sim \mathcal{N}(0, I)$ and an independent data sample $x_1 \sim p_1(x_1)$, we define $z = (x_0, x_1)$. We choose the following path:

$$p_t(x|x_0, x_1) = \delta_{a_t}(x) \tag{2.35}$$

where $a_t = (1 - t)x_0 + tx_1$. It is easy to derive the corresponding vector field:

$$v_t(x|x_0, x_1) = x_1 - x_0 \tag{2.36}$$

With this conditional path, we get the following (still intractable) objective:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],(x_0,x_1)\sim\mathcal{N}(0,I)\times p_1(x_1),x\sim p_t(x|z)}\left[\|v_\theta(x,t) - (x_1 - x_0)\|^2\right] \tag{2.37}$$

Approximating p_1 with p_{data}, we get the following tractable objective (equation (2.34) becomes an approximation):

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],(x_0,x_1)\sim\mathcal{N}(0,I)\times p_{data}(x_1),x\sim p_t(x|z)}\left[\|v_\theta(x,t) - (x_1 - x_0)\|^2\right] \tag{2.38}$$

In this setting, the lines connecting noise to data learned by the model tend to be straight (see the visuals on [8]). This "straightness" is a key advantage of CFM over Diffusion Models, as it allows for much more efficient inference using numerical ODE solvers with large step sizes.

In real settings, people tend to use $p_t(x|x_0, x_1) = \mathcal{N}(x|\mu_t, \sigma_t^2 I)$, where σ_t is small (can be constant) instead of $\sigma_t = 0$, as it leads to better empirical results [17]. We nevertheless used $\sigma_t = 0$ in our experiments to keep the equivalence of Section 4.1.

2.7.5 Choice of the coupling

The pairs (x_0, x_1) need not be drawn from the independent product $p_0 \otimes p_1$. Minibatch optimal-transport couplings (OT-CFM) [17] instead pair each noise sample with the data sample assigned to it by the discrete OT plan computed within the minibatch. This is claimed to straighten the conditional paths and led to faster training on all three datasets.

2.7.6 Summary

Flow Matching in practice:

(i) Choose a variable z with some known distribution $q(z)$
- *e.g.,* $z = (x_0, x_1)$, $q((x_0, x_1)) = p_0(x_0) \times p_1(x_1))$

(ii) Choose a simple conditional distribution $p_t(x|z)$
- *e.g.,* $\delta_{(1-t)x_0+tx_1}(x)$

(iii) Compute an associated velocity field $v_t(x|z)$
- *e.g.,* $x_1 - x_0$

(iv) Train model using conditional loss $\mathcal{L}_{\text{CFM}}$
- *sample* t, z, x *using the data points*

(v) Sample from (a distribution close to) $p_1 \approx p_{data}$
- *sample* $x_0 \sim p_0$*, then use an integration scheme, e.g., Euler Scheme*

A minimal reference implementation of this loop is given in **??**.

3 Related Work

At the start of the internship, the two ingredients of our approach were well developed separately but, to our knowledge, not combined.

On the unrolling side, Le, Repetti and Pustelnik [11] study unfolded proximal networks for robust image Gaussian denoising, establishing the LNO/LFO learning strategies and the associated Lipschitz robustness certificates that we adapt in Section 4.3.2. This line of work, together with the original LISTA construction [9] and the general algorithm-unrolling survey [14], treats UNNs purely as denoisers or inverse-problem solvers: the noise level is either fixed or a nuisance parameter, never a continuously varying conditioning signal driving a generative process.

On the Flow Matching side, the denoising reformulation of Gagneux et al. [8, 7] and the proximal-operator viewpoint of Fukumizu et al. [6] establish, respectively, that the optimal FM velocity is the MMSE denoiser at every noise level (Equation (4.4) below). Both results are stated at the level of the *optimal* velocity field; neither proposes a concrete architecture that is itself proximal. Standard FM implementations, including the `torchcfm` library and its UNet backbones [17] that we use as baselines, instantiate v_θ with generic convolutional networks.

The gap we identified, and that motivates Section 4, is therefore: no existing work instantiates the FM velocity field with a proximal unrolled network, exploits the resulting proximal reading for interpretability, or studies the trade-off against black-box UNet backbones under matched training.

4 Our Contribution

This section presents the contribution of the internship. It is built around a idea that since the optimal Flow Matching velocity field *is* an MMSE denoiser, an unrolled proximal denoiser can be used as a velocity field (Section 4.1), and around the decisions we had to make while turning that idea into a working generative model.

Those decisions are of two kinds, and the section is organised accordingly. Section 4.2 builds the velocity field step by step; each step of the construction describes a degree of freedom that we had to fix, so the subsections there alternate between derivation and decision. Section 4.3 then covers choices that we made empirically, by comparing different versions of our neural networks. Every decision of either kind is flagged in the text by a **Choice** box, and all of them are collected in Table 1, which doubles as a map of the section.

The remainder is implementation: the exact unrolled recursions of the two algorithm families we built on (Section 4.4), the catalogue of models we implemented and trained, including the architectural fix without which convolutional ScCP cannot represent image data at all (Section 4.5), and the training hyperparameters (Section 4.7). Section 4.6 closes the section by contrasting the resulting architecture with a UNet. The numerical results obtained with these models are reported in Section 5, the negative results that shaped the final design and options we did not get the time to explore in Section 8.1.1.

Decision	**Retained**	**Alternative(s) considered**
Fixed throughout the experiments (Section 4.2)		
Role of the UNN	Called at every step of the ODE solver, as a denoiser	One layer = one Euler step, image generated in a single forward pass
Handling $t \to 0$	Warm start from $z = x_t$	Re-deriving the algorithm for an input x_t rather than x_t/t
Handling $t \to 1$	Sample and integrate on $t \in [0, t_{\max}]$	Clamping the denominator inside the network
Output parameterisation	x_1-prediction trained with a velocity-space weighted loss	v-prediction; x_0-prediction; plain x_1 regression
Proximal operator	Analytical ℓ_1 dual prox (clipping, learned radius)	A learned neural module in place of the prox
Chosen via experiments (Section 4.3)		
Algorithm family	ScCP for images	DFB
Parameter constraints	LFO	LNO

Table 1: The decisions taken in this section. The upper block collects the ones we kept during all of our experiments; the lower block the ones that we tested empirically.

4.1 A Denoising Perspective on Flow Matching

In the context of generative models such as Flow Matching, the corrupted observation at time t can be expressed as $x_t = tx_1 + (1-t)x_0$, where $x_0 \sim \mathcal{N}(0, I)$ represents noise. From a denoising perspective, the objective of the model is to estimate the clean signal x_1 from the noisy measurement x_t.

Remark 4.1 We have $x_1 - x_0 = (x_1 - x_t)/(1-t)$.

By rescaling the problem to a standard additive noise model, we can write:

$$z = \frac{x_t}{t} = x_1 + \frac{1-t}{t}x_0 \implies z = x_1 + \epsilon_t, \quad \text{with } \epsilon_t \sim \mathcal{N}(0, \sigma_t^2 I) \tag{4.1}$$

where the effective noise variance is $\sigma_t^2 = \left(\frac{1-t}{t}\right)^2$. It is an inverse problem, with $A = Id$ (see [7]).

Following the Maximum A Posteriori (MAP) framework, the estimate $\hat{x}$ is obtained by maximizing the posterior distribution $p(x_1|z) \propto p(z|x_1)p(x_1)$. Assuming a fixed prior αg, the MAP estimator is the minimizer of the negative log-likelihood:

$$\hat{x} = \arg\min_x \left(\frac{\|z - x\|^2}{2\sigma_t^2} + \alpha g(x) \right) \tag{4.2}$$

By multiplying the objective function by σ_t^2, we recover the standard variational form:

$$\hat{x} = \arg\min_x \left(\frac{1}{2}\|z - x\|^2 + \mu_t g(x) \right), \quad \text{with } \mu_t = \alpha \cdot \sigma_t^2 \tag{4.3}$$

This derivation proves that the optimal regularization parameter μ_t must be proportional to the variance of the noise present at time t.

Equivalently, and in the notation used in the rest of this report, combining $x_t = (1-t)x_0 + tx_1$ with the optimal velocity $v^\star(x_t, t) = \mathbb{E}[x_1 - x_0 \mid x_t, t]$ gives the identity [7, Formula (5)]

$$D_t^\star(x_t) := \mathbb{E}[\, x_1 \mid x_t, t\,] = x_t + (1-t)\, v^\star(x_t, t), \tag{4.4}$$

i.e. the MMSE estimator of the clean image x_1 given the noisy observation x_t and the noise level t. Thus the optimal FM velocity gives the optimal denoiser at every noise level, and conversely any denoiser D_t induces a velocity $v(x,t) = (D_t(x) - x)/(1-t)$. This duality is what motivates the use of denoising-oriented unfolded architectures for FM, and is the central idea of this report.

4.2 Building the Velocity Field with an Unrolled Network

The duality of Section 4.1 says that a denoiser *can* serve as a velocity field; it does not say how to build one. Turning it into an actual network requires four decisions, each forced by a specific step of the construction: where the unrolled network sits with respect to the ODE solver (Section 4.2.1), how the ill-conditioning of the reduction at $t \to 0$ and $t \to 1$ is handled (Section 4.2.2), what the network outputs and against what it is regressed (Section 4.2.3), and how samples are finally produced (Section 4.2.4).

4.2.1 Role of the UNN

Now that we know that we want to use an UNN for FM, two natural options are available to us. *(i)* Each layer of the UNN is interpreted as one Euler step of the generation ODE, so that an image is generated from Gaussian noise in a single forward pass. *(ii)* The UNN is called at every time step of the integrator, and denoises the current noisy image to give the direction in which to move next. We experimented with both and retained option *(ii)*, notably because one of the strengths of velocity modelling comes from using the *same* model to denoise at different noise levels [7].

Choice. The UNN is called at *every* step of the ODE solver and acts as a denoiser at the current noise level, rather than being unrolled once into the whole trajectory. We experimented with both.

4.2.2 Recovering the velocity

As derived in Section 4.1, the vector field estimation at time $t \in]0, 1]$ reduces to solving the following variational problem:

$$\hat{x} = \operatorname*{argmin}_{x \in \mathcal{H}} \frac{1}{2}\|z - x\|^2 + \mu_t\, g(Wx), \tag{4.5}$$

where $z = x_t/t$ is the rescaled noisy observation and $\mu_t = \alpha(1-t)^2/t^2$ is the time-dependent regularisation parameter, with $\alpha > 0$ a learnable global scalar. The velocity field is then recovered via

$$v_\theta(x_t, t) = \frac{\hat{x} - x_t}{1-t}. \tag{4.6}$$

Hence, given a proximal algorithm $\mathcal{A}(\alpha, z)$ solving $\min_x \frac{1}{2}\|z - x\|_2^2 + \alpha g(x)$ (eg. DFB, ScCP, ...), we can use Algorithm 1 to get a proximal algorithm converging to the value of the vector field at any point $(x(t), t)$.

Algorithm 1 Prox_for_vector_field$(\mathcal{A}, x_t, t, \alpha)$

var $\leftarrow ((1-t)/t)^2$ ▷ noise *variance* σ_t^2
$\mu \leftarrow \alpha \cdot$ var
$z \leftarrow x_t/t$
$\hat{x}_1 \leftarrow \mathcal{A}(\mu, z)$
return $(\hat{x}_1 - x_t)/(1-t)$

As simple as this derivation may be, two numerical subtleties arise from this denoising reduction. First, the rescaling $z = x_t/t$ of Algorithm 1 blows up as $t \to 0$. In practice we cannot divide by t in our code. We therefore warm-start every unrolled network directly from $z = x_t$ (instead of xt/t), which does not blow up near 0. This seemed to us the simplest solution to our issue and has the advantage of allowing us to use our algorithm for any model based on a classical denoiser (ie. a denoiser designed to take as input $z = x^* + \epsilon$ and recover x^*), but has the downside of shifting us away from the mathematical foundations of our algorithm, because we just remove a division and let the network figure out what to do with its input. Another solution would have been to re-derive 4.1 and work on a different version of the ScCP algorithm designed to take x_t as input. More on this in section [???].

Second, the final division by $1-t$ blows up as $t \to 1$; this issue is simpler to solve: at inference as well as during training, we always sample the time t in $[0, t_{max}]$ (because we are using Euler discretization to solve the ODE). So the denominator is never too small, following [7, Section 3.3], [12]. We also experimented with applying $max(., t_{max})$ on the denominator inside the network, which led to similar results, but does not maintain the equivalence between the denoising and the velocity modeling, so it was not preferred.

Choice. Near $t = 0$ we warm-start the unrolled network from $z = x_t$ rather than $z = x_t/t$, letting the learned operators deal with the $1/t$ factor; near $t = 1$ we sample and integrate on $t \in [0, t_{\max}]$ rather than clamping the denominator inside the network. Both keep the network well conditioned. Our first choice implies some distance with the mathematical derivation.

4.2.3 Output parameterisation: x-prediction with a velocity-based loss

We can go further than the duality shown in Section 4.1. A model estimating the vector field can be parameterised in three equivalent ways: predicting the velocity v itself, the denoised image x_1, or the source noise x_0. It is unclear in the literature which is best in general.

In *velocity prediction* (v-pred) the network outputs the velocity directly, $v_\theta(x_t, t)$, and is trained with the standard CFM loss $\|v_\theta - (x_1 - x_0)\|^2$. In *clean-signal prediction* (x-pred) the network instead outputs an estimate $D_\theta(x_t, t) \approx x_1$ of the clean image, and the velocity is reconstructed *a posteriori* via $v_\theta(x_t, t) = (D_\theta(x_t, t) - x_t)/(1-t)$. This is the natural output of our unrolled solvers, whose primal iterate $\hat{x}$ is by construction an estimate of x_1 (the MAP denoiser of Section 4.1).

The two parameterisations are made equivalent by the choice of loss. Using $x_1 - x_t = (1-t)(x_1 - x_0)$, the velocity error factorises as

$$\left\|v_\theta(x_t, t) - (x_1 - x_0)\right\|^2 = \frac{1}{(1-t)^2}\left\|D_\theta(x_t, t) - x_1\right\|^2. \tag{4.7}$$

Hence, rather than regressing D_θ onto x_1 with a plain $\|D_\theta - x_1\|^2$ objective, we train it with the *velocity-based weighted* loss

$$\mathcal{L}_{\text{x-pred}}(\theta) = \mathbb{E}_{t,x_0,x_1}\left[w(t)\left\|D_\theta(x_t, t) - x_1\right\|^2\right], \qquad w(t) = \frac{1}{(1-t)^2}, \tag{4.8}$$

This does not explode because $t \leqslant t_{max}$ (see Section 4.2.2).

By (4.7), this is *identical* to the CFM objective $\mathbb{E}\|v_\theta - (x_1 - x_0)\|^2$: the network is optimised for the correct velocity-matching goal, while regressing a target (x_1) that is bounded, well scaled and image-like at all times. We use x-prediction and a velocity-based loss because the architecture used here has its roots in image denoising, and following the results of [7] and [12].

Choice. The network predicts the clean signal x_1, not the velocity v nor the noise x_0, and is trained with the velocity-space weighted loss (4.8) rather than a plain $\|D_\theta - x_1\|^2$ regression. The first half of the choice follows the architecture, which is a denoiser; the second half is makes it equivalent to the standard CFM objective.

4.2.4 Inference

Once we have trained an UNN to model the vector field, at inference, we draw $x \sim \mathcal{N}(0, I)$ and integrate $\dot{x}(t) = v_\theta(x(t), t)$ with an explicit Euler scheme over N ($= 20$, so $t_{max} = 0.95$) steps. At each step, the current iterate plays the role of the noisy observation of (4.5), i.e. we set $z = x_t$. The noise level is passed separately through t, on which the proximal module $\mathcal{N}_{\theta_{\text{prox}}}$ is conditioned. The network returns $\hat{x}_1 = D_\theta(x_t, t)$ after K unrolled iterations, and the velocity is recovered through (4.4).

4.3 Architectural Design Choices

The construction of Section 4.2 pins down what the network must compute and how it is trained. What it leaves open is the unrolled architecture itself: which proximal algorithm is unrolled, how its parameters are constrained, and what plays the role of the proximal operator. These are the axes we explored experimentally.

4.3.1 Choice of the Algorithm Family

The variational problem (4.5) can be solved by any of the proximal algorithms of Section 2 (and more), and we built UNNs from two of them: DFB, the simplest scheme, and ScCP, which exploits the strong convexity of the data-fidelity term $f_z(x) = \frac{1}{2}\|z - x\|^2$ to obtain an accelerated rate. Both are evaluated on *two moons* (Section 5.1), where ScCP does not improve over plain DFB despite its stronger guarantees.

Choice. After this experiment on 2-moons, we nonetheless kept only *ScCP* for images, because we expected better results, based on prior papers **??**.

4.3.2 LNO vs. LFO

Following [11], every UNN is trained under two learning strategies for its parameter set $\Theta = \{W_k, V_k, \tau_k, \sigma_k, \alpha, \ldots\}$, where V_k is the learned counterpart of $W_k^\top$:

(i) **Learned Normalised Operators (LNO).** Theoretical convergence conditions are enforced at each layer: the adjoint is exact ($V_k = W_k^\top$) and stepsizes are bounded by the spectral norm of W_k, computed via power iterations, but independent.

(ii) **Learned Flexible Operators (LFO).** The adjoint constraint is relaxed ($V_k \neq W_k^\top$), and stepsizes are dependent, but learned without spectral constraints.

For ScCP, the flexibility of LFO lies in the *operators* rather than in the step-sizes. It is *LNO* that learns each primal step-size τ_k independently, compensating through the per-layer normalisation $\sigma_k = 0.99\,(\tau_k \|W_k\|_S^2)^{-1}$, whereas LFO ties the entire schedule $(\tau_k, \sigma_k, \alpha_k)$ to the single scalar τ_0 through the acceleration recursion (4.12) (see Section 4.4 for more details).

Choice. We use *LFO* in the final image models. The two regimes are close empirically, with LFO similar or better (Section 5.1); the theoretical guarantees of LNO did not translate into a measurable advantage here.

4.3.3 Choice of Proximal Operator

In our experiments, the dual proximal operator is fixed to the analytical form induced by the ℓ_1-norm prior. Since $(\mu\|\cdot\|_1)^* = \delta_{\mathcal{B}_\infty(\mu)}$, the dual proximal step reduces to a clamping operation:

$$\mathrm{prox}_{\tau(\mu\|\cdot\|_1)^*}(u) = \mathrm{clamp}(u,\, -r(t),\, r(t)), \tag{4.9}$$

where $r(t) = \mathrm{softplus}(\mathrm{MLP}(t))$ is a learned, time-dependent radius (module `L1ProxFlat` for flat models, `L1ProxConv` for convolutional ones).

Choice. The dual proximal operator is kept in its *analytical* ℓ_1 form, rather than replaced by a learned neural module. The expressivity of the model is then carried by the operators W_k and V_k rather than by the nonlinearity, which is what preserves the proximal reading of the layer. Together with the bias of Section 4.5.4, the implemented operator is `clamp(· + b, MLP(t))`.

4.4 Algorithm Parameterisations

We now detail the two algorithm families used as the backbone of our UNNs, together with the exact parameterisations adopted in the LNO and LFO regimes.

4.4.1 Dual Forward-Backward (DFB)

The DFB unrolled iterations read, for $k = 0, 1, \ldots, K-1$:

$$\begin{cases} x_{k+1} = z - V_k\, u_k, \\ u_{k+1} = \mathcal{N}_{\theta_{\mathrm{prox}}}(u_k + \tau_k W_k x_{k+1},\ t)\,, \end{cases} \tag{4.10}$$

initialised with $x_0 = z$ and $u_0 = 0$.

DFB-LFO. The learnable parameters per layer are $\Theta_k = \{W_k, V_k, \tau_k, \theta_{\mathrm{prox},k}\}$, where W_k and V_k are independent (mismatched adjoint), and $\tau_k = \mathrm{softplus}(\hat{\tau}_k)$ with $\hat{\tau}_k$ initialised at 0.5.

DFB-LNO. The adjoint is matched: $V_k = W_k^\top$. The stepsize is fixed to $\tau_k = 1.99\,\|W_k\|_S^{-2}$, where $\|W_k\|_S$ is estimated by power iteration at each forward pass ($n_{\mathrm{iter}} = 3$).

4.4.2 Strongly-Convex Chambolle–Pock (ScCP)

The ScCP scheme exploits the strong convexity of the data-fidelity term $f_z(x) = \frac{1}{2}\|z - x\|^2$ to derive an accelerated primal-dual algorithm. Its iterations read:

$$\begin{cases} x_{k+1} = \dfrac{x_k - \tau_k V_k u_k + \tau_k z}{1 + \tau_k}, \\ y_{k+1} = x_{k+1} + \alpha_k(x_{k+1} - x_k), \\ u_{k+1} = \mathcal{N}_{\theta_{\mathrm{prox}}}(u_k + \sigma_k W_k y_{k+1},\ t)\,, \end{cases} \tag{4.11}$$

initialised with $x_0 = z$, $u_0 = 0$. The momentum α_k and the schedule of (τ_k, σ_k) follow the accelerated recursion, with the strong-convexity constant of f_z fixed to 1:

$$\alpha_k = (1 + 2\,\tau_k)^{-1/2}\,, \qquad \tau_{k+1} = \alpha_k \tau_k, \qquad \sigma_{k+1} = \frac{\sigma_k}{\alpha_k}. \tag{4.12}$$

ScCP-LFO. Only τ_0 is learned (initialised via $\tau_0 = \mathrm{softplus}(\hat{\tau}_0)$, $\hat{\tau}_0 = -0.5$); the full $(\tau_k, \sigma_k, \alpha_k)$ sequence is then derived recursively from (4.12). W_k and V_k are learned independently; $\sigma_0 = 1$ (absorbed into W_k).

ScCP-LNO. Each τ_k is learned independently, $\tau_k = \mathrm{softplus}(\hat{\tau}_k)$; $V_k = W_k^\top$; and $\sigma_k = 0.99\,(\tau_k\,\|W_k\|_S^2)^{-1}$, ensuring the product $\tau_k\sigma_k\|W_k\|_S^2 = 0.99 < 1$ at every layer, so that each layer is a valid ScCP step in isolation. The momentum α_k is derived from τ_k as above.

4.5 Implemented Models

We now list the models that were actually implemented and trained. All UNN architectures share the following design conventions:

- **Time conditioning.** If the proximal operator is an MLP, t is concatenated directly to the input of the MLP. For convolutional or ℓ_1 proximal operator, a small MLP maps $t \mapsto$ a channel-wise bias.
- **Truncation depth.** $K \in \{3, 5, 10, 15\}$ for our models.

4.5.1 Baselines

Four reference deep-learning models are included:

- `MLP_baseline`: time-conditioned MLP, hidden width $w = 1024$.
- `UNet_torchCFM_baseline`: built-in UNet from the `torchcfm` library.
- `UNet_baseline`: custom UNet, base channel count `base_ch` $= 32$.
- `SmallUNet_baseline`: lighter UNet, `base_ch` $= 32$.

4.5.2 Standard UNNs

Each iteration k has its own independent parameters $\{W_k \in \mathbb{R}^{d_u \times d}, V_k \in \mathbb{R}^{d \times d_u}, \tau_k, \sigma_k, \theta_{\text{prox},k}\}$, with $d = \dim(\mathcal{H})$ and $d_u = d$ (same-dimension dual). The proximal operator is a small time-conditioned MLP with hidden width $w = 32$. Two families are evaluated: `DFB_UNN` and `ScCP_UNN`, each in LFO and LNO versions.

4.5.3 Convolutional UNNs

For image data the dense operator W_k is replaced by a learnable 2D convolution and V_k by a transposed convolution of matching shape. Concretely, W_k maps the primal image (1 channel for MNIST, 3 for CIFAR-10) to a $d_u =$ `ic`-channel dual feature space with a 9×9 kernel and padding 4 (so that `conv_transpose2d` is the exact adjoint of `conv2d` and spatial size is preserved), where the number of intermediate channels `ic` is a hyperparameter. The convolution weights are Kaiming-initialised. Unless stated otherwise the proximal operator is always the analytical ℓ_1 dual prox (`L1ProxConv`). Two families are provided: `ConvDFB_UNN` and `ConvScCP_UNN`, each in LFO and LNO versions.

4.5.4 Breaking the odd symmetry of the proximal velocity field

The convolutional ScCP block with the analytical ℓ_1 dual prox has a structural property that is invisible on the (centred, near-symmetric) two-moons data but harmful on images: as a function of its input, the induced velocity field is *odd*. Indeed, with warm start $z = x_t$, dual initialisation $u_0 = 0$, and no convolutional bias, every ScCP update (primal step, extrapolation, dual forward step) is *linear* in (x, u, z), and the symmetric clipping prox satisfies $\text{prox}(-a) = -\text{prox}(a)$. By induction the map $x_t \mapsto \hat{x}$ is odd, hence

$$v_\theta(-x_t, t) = -\, v_\theta(x_t, t). \tag{4.13}$$

Property (4.13) propagates through the generative ODE: $\text{gen}(-x_0) = -\text{gen}(x_0)$. On (normalized) MNIST, a digit has a black background (≈ -1) and white strokes, so its negation is a *colour-inverted* image (white background). Since the noise x_0 and $-x_0$ are equally likely, the generator is forced to produce a distribution that is symmetric about 0: roughly half of the generated samples come out colour-inverted, and the model cannot represent the asymmetric MNIST distribution. This is a genuine limitation inherited from the odd symmetry of the underlying convex/proximal problem.

We break the symmetry with a **learned convolutional bias (`w_bias`)**. A per-dual-channel bias b is added in the dual forward operator, $\sigma_k\,(W_k y_{k+1} + b)$, which amounts to penalising $g(W_k \cdot - b)$ instead of $g(W_k \cdot)$. The shift makes the map non-odd in x_t. With b zero-initialised the model starts exactly at the symmetric operator.

This fix preserves the proximal/variational interpretation of the layer (it corresponds to a shifted argument) while removing the parity constraint that made unmodified ScCP unable to fit image data.

4.6 Structural Differences with a UNet

Our convolutional ScCP differs from a UNet in four ways, all three of which matter for the interpretation of **??**.

No multiscale. A UNet builds its receptive field by successive downsampling, so intermediate activations live on coarser grids and are no longer images. ScCP has no pooling: the primal iterate x_k always stays at full resolution, and its receptive field grows only *linearly* with the number of unrolled iterations K. This is what makes the internal states directly displayable. This locality does not appear to limit generation as long as the network has enough layers.

Primal–dual alternation with input re-injection. A UNet propagates a single stream of feature maps, with skip connections carrying resolution across the encoder–decoder. ScCP instead alternates between a few-channel primal variable in image space (one channel for MNIST, three for CIFAR-10) and a multi-channel dual variable, and re-injects the observation z at *every* primal step (see (4.11)). The architecture therefore anchors the primal variable to the input at each step, rather than progressively constructing an abstracted representation of it.

Nonlinearity. The only nonlinearity in ScCP is a proximal operator — here a clipping with a learned, time-dependent radius — whereas a UNet stacks pointwise activations together with normalisation layers, which aggregate spatial statistics over the whole feature map. Every ScCP layer is thus a strictly local, translation-equivariant operation.

No attention and no batchnorm. In the torchCFM UNet, contrary to our models, Attention Blocks are used to improve the performance of the architecture. The torchCFM UNet was used as an objective to aim for, but not necesseraly to attain, because of this kind of differences.

4.7 Hyperparameters

All models are optimised with Adam (learning rate 10^{-3}) via the standard CFM loss $\mathcal{L}_{\mathrm{CFM}}(\theta)$, using Minibatch Optimal Transport couplings [17]. Dataset-specific settings are as follows: batch size 256 and 20 000 iterations on *two moons*; batch size 128 on MNIST [TODO - experiments based].

5 Numerical Experiments

We evaluate the architectures of Section 4 against standard deep-learning baselines (MLP, UNet) on *two moons*, MNIST (28×28 greyscale) and CIFAR-10 (32×32 RGB).

5.1 Results on *Two Moons*

Figures 1 and 2 report training loss curves of ScCP LNO/LFO and DFB against an MLP baseline over 50 epochs on 10000 images, as well as the performance of ScCP LNO after 50 epochs. All the model analysed achieved good performance on 2-moons, as you can see in Table 2

5.2 Results on MNIST

From two moons to images. Moving to the 28×28 image space of MNIST requires adapting the architectural inductive biases: dense operators scale poorly and ignore spatial locality. As described in Section 4.5.3, we replace the linear operator W_k by a convolution mapping the 1-channel primal image to an `ic`-channel dual feature space (9×9 kernel, padding 4). Our MNIST study focuses on a **single** model family: the convolutional strongly-convex Chambolle–Pock network `ConvScCP_UNN` with the **analytical** ℓ_1 **dual prox**, trained in the x**-prediction / velocity-loss**

Dataset	Model	Config.	#Params	Iterations	Val. loss	FID
2-Moons	ScCP LNO	K=10, m=32	1 940	500k	?	–
	ScCP LFO	K=10, m=32	2 571	500k	?	–
	DFB LNO	K=10, m=32	?	500k	?	–
	DFB LFO	K=10, m=32	?	500k	?	–
	MLP	–	4 546	500k	?	–
MNIST	ScCP LNO	K=20, m=64	105 480	46 900	?	–
	ScCP LFO	K=20, m=64	–	46 900	–	–
	SmallUNet	–	–	46 900	–	–
	train vs. test reference	10k/10k imgs	–	–	–	1.03
CIFAR-10	ScCP LFO	K=20, m=512	–	180 000	–	–
	UNet (torchcfm)	–	–	400 000	–	–

Table 2: Quantitative comparison across the three datasets. All runs use OT couplings (Section 2.7.5) and the x-prediction / velocity-loss parameterisation of Section 4.2.3. The MNIST FID is computed in the feature space of a small MNIST classifier, not InceptionV3 (see **??**). [TODO: complete missing entries; add GFLOPs/fwd and training time columns once measured.]

setting of Section 4.2.3, operating directly in pixel space. The proximal operator is thus fully interpretable (a per-channel ℓ_1 clipping with learned time-dependent radius), and the only black-box component is the linear analysis operator W_k.

Experimental setting. We first train `ConvScCP_UNN` in the LNO regime on the single-class subset of MNIST restricted to the digit 0 (batch size 128, 50 epochs, Adam). We sweep the number of unrolled iterations $K \in \{5, 10, 20\}$ and the number of intermediate channels `ic` $\in \{32, 64, 128\}$, and compare against `SmallUNet` baselines of comparable or larger parameter count trained under the same Flow Matching objective. This sweep predates the coupling fix reported in Section 8.1.1 and therefore uses independent coupling; the full-MNIST results of Table 2 use OT coupling.

Interpretability of the trajectories The role of the UNN is to denoise the image at each time step t. Hence, two kinds of denoising happen in the generation of an image: the "global denoising" corresponding to the generation, as seen in the first column of Figures 5 and 6, and a "local denoising" happening inside the UNN at each time step, which can be seen in the rows of Figures 5 and 6. ScCP seems to denoise the image little by little, where it could in principle use the intermediate images only to store information as in CNN channels. We believe this is because of the re-injection of z in the primal step in (2.19). This is true for both ScCP LFO and LNO, even though something else happens around layer 5 in ScCP LNO. We conjecture this is because in LNO, the primal step-sizes τ_k are learned independently, while the stability constraint ties $\sigma_k = 0.99\,(\tau_k \|W_k\|_S^2)^{-1}$ to them. Whenever $\tau_k \|W_k\|_S$ becomes small at some layer, σ_k (and hence the dual amplitudes) becomes very large.

5.3 Results on AFHQ

As explained in Section 8.2.2, it was hard to experiment on more complex datasets, mainly due to the compute time needed to train our architectures, as they were not well adapted to the structure of the GPUs. To generate colored images, we started by experimenting on CIFAR-10, and then pivoted to AFHQ-32 to tackle a simpler problem. Even here, we did not have the time to get definitive results. Table 2 describes a comparison between our baseline and the best model we were able to train. See Figure 7 for a visual comparison. Overall, we had some motivating results and our models seem to be able to learn the right velocity field on AFHQ-32, but at the cost of a long training time (30h of compute to train a model, where a comparable UNet would need 5h of training).

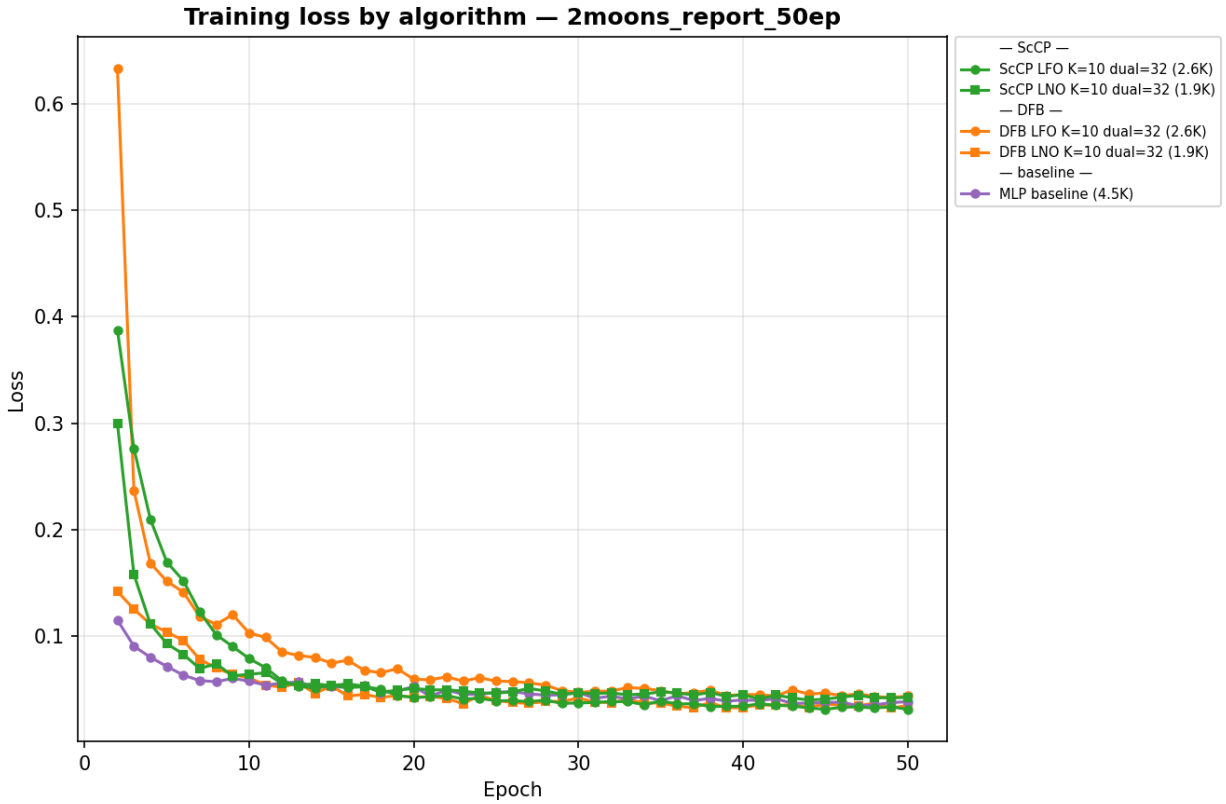


Figure 1: Training loss on *two moons* grouped by algorithm family (ScCP in green, DFB in orange). The MLP baseline (4.5K parameters) is shown in purple.

6 Conclusion and Future Work

7 Personal Remarks

8 Appendix

8.1 Explored Directions

The architecture reported in Section 5 is the end point of a much wider exploration. This section documents that exploration: the dead ends that shaped the final design, the side investigations that answered a question without changing the architecture, and the questions we opened but could not close.

Throughout, each item ends with an explicit verdict, using the following convention: *confirmed* means measured and we got a positive result; *refuted* means the hypothesis was ours (or the literature's) and the measurement contradicted it; *inconclusive* means the measurement was made but does not support a conclusion at the current sample size or budget; and *conjecture* marks a reading we have not tested. Unless stated otherwise, all numbers below are single runs, which is the main limitation of this section.

8.1.1 What did not work

Odd symmetry of the proximal field. The plain ℓ_1 ConvScCP is *exactly* an odd operator, so half of the generated MNIST samples came out colour-inverted (Section 4.5.4). This is architectural, holds at initialisation, and is not an optimisation artefact. We tried two zero-initialised fixes: a learned convolutional bias in the analysis operator (`w_bias`), and an asymmetric two-threshold dual prox $\mathrm{clamp}(u, -\lambda^-, \lambda^+)$, which is the exact prox of $\lambda^+ \max(x, 0) + \lambda^- \max(-x, 0)$ and therefore also preserves the proximal reading. Both break the parity, but we kept only `w_bias`: it acts on the input dependence of the map rather than on a static threshold, and it leaves a single mechanism to explain in the paper. *Verdict: confirmed (the symmetry), fixed (the model).*

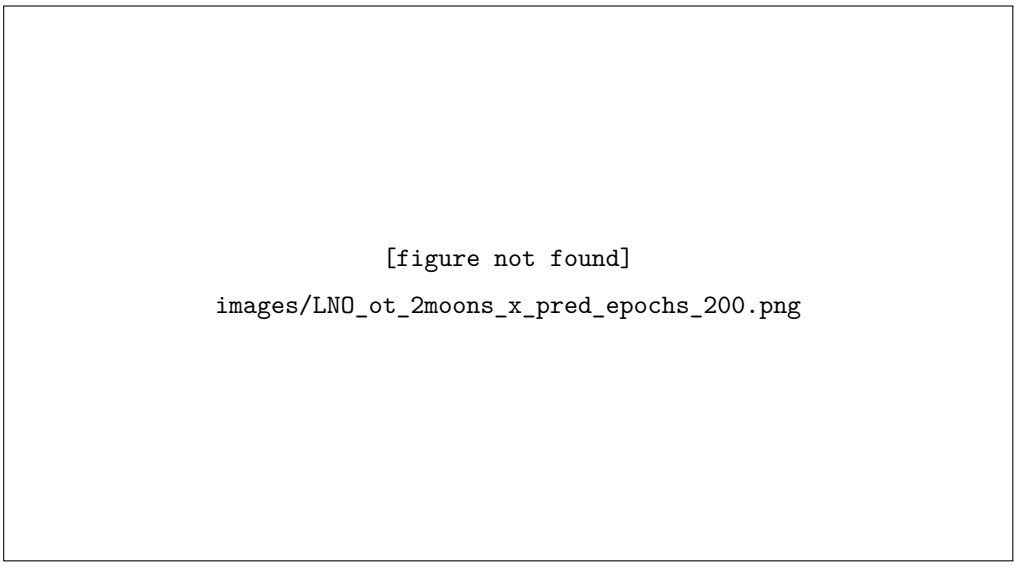


Figure 2: Generation of the two moons by ScCP LNO after 200 epochs, with $K = 10$ layers and a dual dimension $m = 32$. [TODO: regenerate with larger axis labels — currently too small to be read.]

Latent-space Flow Matching. We tried running the unrolled flow in the latent space of a light convolutional (mildly variational) autoencoder: each 28×28 image is encoded to a $(c_{\text{lat}}, 7, 7)$ code, the flow is matched on the codes, and the decoder is used only for display. This was tried because we wanted to make it easier for the ScCP to mix information between channels as in a UNet (Section 4.6). It consistently produced blurrier digits than the pixel-space model, and the ScCP iterations converged worse. Our reading, offered as a *likely cause* rather than a measured fact, is that the analytical ℓ_1 dual prox is no longer an appropriate prior in that domain: sparsity is a meaningful regulariser on a pixel or wavelet representation, but autoencoder codes are dense entangled features for which an ℓ_1 clipping is not a meaningful proximal step, so the unrolled solver loses the variational structure it relies on. A second objection is that this design moves the credit for sample quality from the proximal architecture to the decoder, which defeats the purpose of the study. *Verdict: abandoned; all reported results are in pixel space. Conjecture:* a structured latent (wavelet or sparse-coding compatible) rather than a learned dense one would be the way to retry this.

Weight sharing and the choice of algorithm family. We tried tying $(W, V, \tau, \sigma, \theta_{\text{prox}})$ across the K unrolled iterations. This allowed a variable inference depth, but costs a large and consistent amount of final loss on two moons (shared models stagnate above 2.1 where per-layer models reach $[1.75, 1.95]$) and does not recover on MNIST. *Verdict: confirmed (gap in the loss); we abandoned shared UNNs.*

8.1.2 Warm-starting the dual variable across ODE steps

At sampling time, each velocity evaluation restarts the unrolled recursion from $u^{(0)} = 0$, so the dual state built at the previous Euler step is discarded. Since consecutive ODE steps solve nearly identical denoising problems, carrying u forward is the natural warm start. It is something a UNet cannot do. The issue with this idea is that we need to find a way to train the model with u_{prev}, which is not obvious in the CFM training method. We implemented it through the latent self-conditioning trick of RIN [10]: with probability 0.9, a no-grad cold pass on the previous state supplies u_{prev}. The idea is that 0.9% of the time, during training, $u^{(0)}$ is defined as the output of a first forward pass of the model (hence $^{(0)} \sim u_{prev}$).

The results we got seemed to indicate that warm-starting compensated when we lacked layers, but was of no use when the model had enough layers. Indeed, the result depends entirely on K. On MNIST (digit 0, 200 epochs, OT coupling, mini-FID of **??**, whose floor is ≈ 4):

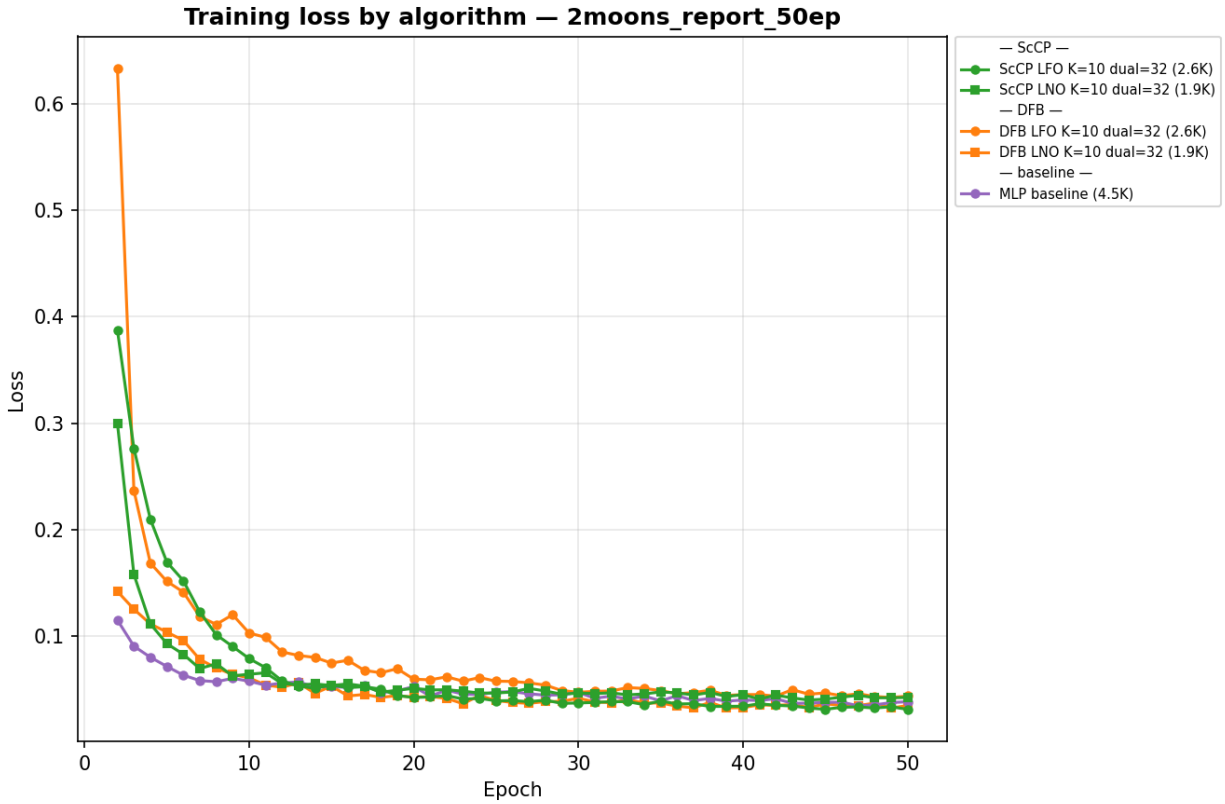


Figure 3: Training loss on *MNIST* grouped by algorithm family (ScCP in green). The UNet baseline (4.5K parameters) is shown in purple.

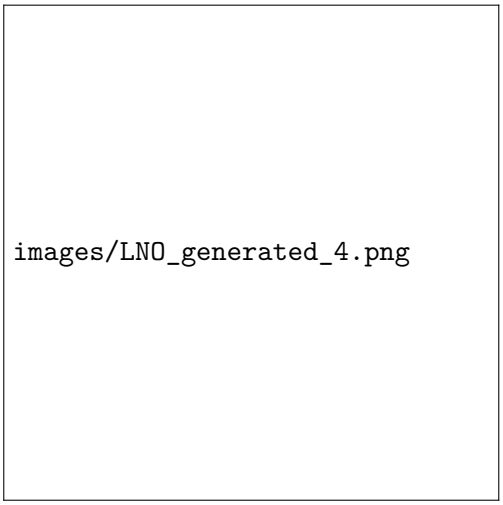


Figure 4: Performance of ScCP LNO on MNIST, with 20 layers and a dual dimension of 64.

Figure 5: Intermediate steps in ScCP LFO on MNIST. Images are normalized; $\max |x_k|$ is written under every image.

Figure 6: Intermediate steps in ScCP LNO on MNIST. Images are normalized; $\max |x_k|$ is written under every image.

Figure 7: Intermediate steps in ScCP LNO on MNIST. Images are normalized; $\max |x_k|$ is written under every image.

K	baseline / cold	self-conditioned / warm	effect
20	23.8	57.3	-141%
6	86.2	64.0	$+26\%$
3	301.5	1059.1	-251%

At $K = 20$, the unrolled recursion converges to the same dual whatever it is initialised with. Tracking the relative step-to-step change of the converged dual along the sampling trajectory, $\|u_K^{(n)} - u_K^{(n-1)}\| / \|u_K^{(n-1)}\|$, the cold and warm regimes start far apart (2.1×10^{-2} against 9.1×10^{-2} at the first step, for the baseline) and collapse onto each other by the end of the trajectory (8.47×10^{-3} against 8.46×10^{-3}); the self-conditioned model behaves the same way at a slightly lower level (6.67×10^{-3} against 6.65×10^{-3}), having learned a marginally more contractive recursion. We first read the degradation as exposure bias — the model sees a cold u_{prev} during training and a warm one at inference — and refuted it with a third sampler that recomputes u_{prev} by a cold pass on the previous state, reproducing the training condition exactly: it gives 57.31, the same value to two decimals as the recursive sampler, i.e. the recursion adds nothing. At $K = 6$ the transfer does pay ($86.2 \to 64.0$, a 26% gain) — but the same control shows this comes from *conditioning* on a dual computed at the previous state, not from chaining along the trajectory: the control sampler reaches 58.6, better still than the 64.0 of the recursion. And even at its best, warm starting never beats the cheaper option of unrolling further: 64.0 at $K = 6$ against 23.8 for the plain $K = 20$ baseline. On AFHQ ($K = 10$, an open channel by the same criterion) the paired run is blurrier and less diverse in both regimes. *Verdict: unconclusive*; *confirmed* that warm-starting can bring worse results when used with sufficiently many layers.

8.1.3 Different choices of proximal operators

We experimented with many proximal operators, namely the $\ell 1$ proximal operator we use in all experiments of Section 5, but also a `DoubleConvTime` consisting of two convolutions with `GroupNorm` and `SiLU`, as well as a small UNet or an MLP as a proximal operator. Those were abandoned to focus on a simple case, keep the proximal interpretation of this part of the architecture and keep the expressivity of the model in the matrices V and W. *Verdict: unconclusive.* This was abandoned to lock in a few degrees of freedom and focus on a simple case.

8.2 A prototyping bench: denoising at fixed noise levels

Full generative training is too slow on big datasets (see Section 8.2.1), so we built a cheap surrogate: the same FM objective restricted to a finite set of noise levels t, which is exactly a denoising benchmark, plus an optimal *linear* (Wiener) denoiser measured on the same split as a reference. Two lessons came out of it.

First, *Our simple models did not achieve as good performance as the reference UNets on complex datasets.* ScCP (1.25M, 2h09) scores 0.2086 while a modern residual UNet that is *smaller* (1.11M) reaches 0.1929 in 21 min.

Second, *successful denoising at fixed time is not the same as successful flow matching.* The same ScCP architectures which were not able to provide good results for flow matching were still pretty good at denoising at fixed time, especially once $t > 0.5$. This complicated our task, because we had to wait for full generative training to compare and choose the best architectures.

8.2.1 The computational cost of unrolling

Our wondered why ConvScCP training runs take roughly seven times longer than a comparable UNet. The answer is neither the parameter count nor the arithmetic: ScCP at $K = 20$ costs 21 GFLOPs per velocity evaluation, 1.5 times the `SmallUNet` and 6.5 times *less* than the torchcfm UNet that trains at the same speed. It is GPU efficiency, about five times worse: on MNIST, the unrolled recursion decomposes the computation into ≈ 2000 tiny CUDA kernels (against 213 for `SmallUNet`), all at full 28×28 resolution since there is no downsampling, with low-arithmetic-intensity 9×9 convolutions between 1 and `ic` channels and elementwise primal–dual algebra on the

dual variable at every iteration. For example, dividing the kernel size from 9 to 3 divides the FLOPs by eight but the runtime by only 1.5. A corollary worth stating, since it misled us: `nvidia-smi` reported 99% utilisation throughout, but that field measures the fraction of time during which at least one kernel is resident and is blind to throughput — ScCP and `SmallUNet` show the same SM occupancy (80% against 82%) at 129 against 620 GFLOP/s. `torch.compile` did help, but not significantly. *Verdict: confirmed; this is a structural property of unrolled architectures on current hardware, and it bounds what an internship-scale compute budget can explore.*

8.2.2 Scaling up: CIFAR-10 and AFHQ-32

The natural next dataset after MNIST was CIFAR-10, with the OT-CFM UNet [17] as the reference backbone. However, a ConvScCP sized for CIFAR (36.5M parameters) ran at 0.22 it/s, i.e. an ETA of 514 hours. We therefore pivoted to AFHQ-cats at 32×32, a single-mode aligned distribution of 5 653 images that is 20 times cheaper per step. Here are our three findings on this dataset.

The transport is learned long before the sharpness. Comparing our 50-epoch UNet with the official 400k-step OT-CFM weights from the *same* x_0, both produce the *same scene* (the same car, the same dog, the same composition) and differ only in sharpness. *Verdict: confirmed, single comparison.*

Below a threshold in steps, a run is not evaluable. On AFHQ-32, incomplete runs (45–55k steps) produce blobs while the one run taken to 165k steps produces genuine cats, at *lower* capacity than two of the failing runs. A delayed-start diagnostic, ie. integrating from a real x_{t_0} and locating the $t_0^\star$ below which samples stop being sharp, localises the failure: the field is learned from $t = 1$ backwards, and the interval $[0, 0.5]$, which decides global structure, converges last. The two *largest* models (9.6M and 10.4M) are the worst at a fixed step budget: capacity is counter-productive when it is not paid for in steps. A train/validation denoising comparison confirms the reading, with a generalisation gap statistically indistinguishable from zero on four checkpoints (including one at 2500 equivalent epochs): these models *underfit*, so adding data would change nothing. *Verdict: confirmed* — with the practical rule that no AFHQ run below $\approx$ 150k steps should be used to conclude anything about an architecture. However, given the time needed to make those runs (often at least 20-30 hours to train one version of ScCP), we do not have extensive results concerning AFHQ-32.

References

[1] Kenneth J. Arrow, Leonid Hurwicz, and Hirofumi Uzawa. *Studies in Linear and Non-Linear Programming*. Stanford University Press, Stanford, CA, 1958.

[2] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM J. Imaging Sci.*, 2(1):183–202, 2009.

[3] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.

[4] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[5] Patrick L. Combettes and Valérie R. Wajs. Signal recovery by proximal forward-backward splitting. *Multiscale Modeling & Simulation*, 4(4):1168–1200, 2005.

[6] Kenji Fukumizu, Wei Huang, Han Bao, Shuntuo Xu, and Nisha Chandramoorthy. Flow matching from viewpoint of proximal operators, 2026.

[7] Anne Gagneux, Ségolène Martin, Rémi Gribonval, and Mathurin Massias. The generation phases of flow matching: a denoising perspective. *arXiv preprint arXiv:2510.24830*, 2025.

[8] Anne Gagneux, Ségolène Martin, Rémi Emonet, Quentin Bertrand, and Mathurin Massias. A visual dive into conditional flow matching. In *ICLR Blogposts 2025*, 2025. https://dl.heeere.com/conditional-flow-matching/blog/conditional-flow-matching/.

[9] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th international conference on international conference on machine learning*, pages 399–406, 2010.

[10] Allan Jabri, David J. Fleet, and Ting Chen. Scalable adaptive computation for iterative generation. In *International Conference on Machine Learning (ICML)*, PMLR, 2023.

[11] H.T.V. Le, A. Repetti, and N. Pustelnik. Unfolded proximal neural networks for robust image Gaussian denoising. *IEEE Trans. Image Process.*, 33:4475–4487, 2024.

[12] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 36115–36125, 2026.

[13] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023.

[14] Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2):18–44, 2021.

[15] Jean-Christophe Pesquet, Audrey Repetti, Matthieu Terris, and Yves Wiaux. Learning maximally monotone operators for image recovery. *SIAM Journal on Imaging Sciences*, 14(3):1206–1237, 2021.

[16] Audrey Repetti. Proximal neural networks: Marrying variational methods and artificial intelligence. EUSIPCO tutorial, 2025. `https://sites.google.com/view/audreyrepetti/events/2025-eusipco-tutorial`.

[17] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024.